

TABLE OF CONTENTS

1. Introduction
2. Pre Requisites
3. Project Objectives
4. Project Flow
5. Project Structure
6. Data Collection
 - Download The Dataset
7. Data Pre-Processing
 - Import Necessary Libraries
 - Importing The Dataset
 - Analyze The Data
 - Handling Missing Values
 - Data Visualization
 - Splitting The Dataset into Dependent and Independent Variable
 - Feature Scaling
 - Splitting The Data into Train and Test
8. Model Building
 - Training and Testing the Model
 - Model Evaluation
 - Save The Model
9. Application Building
 - Build HTML Code
 - Main Python Script
 - Run the App
 - Output
10. Conclusion

1. INTRODUCTION

TrafficTelligence: Advanced Traffic Volume Estimation with Machine Learning

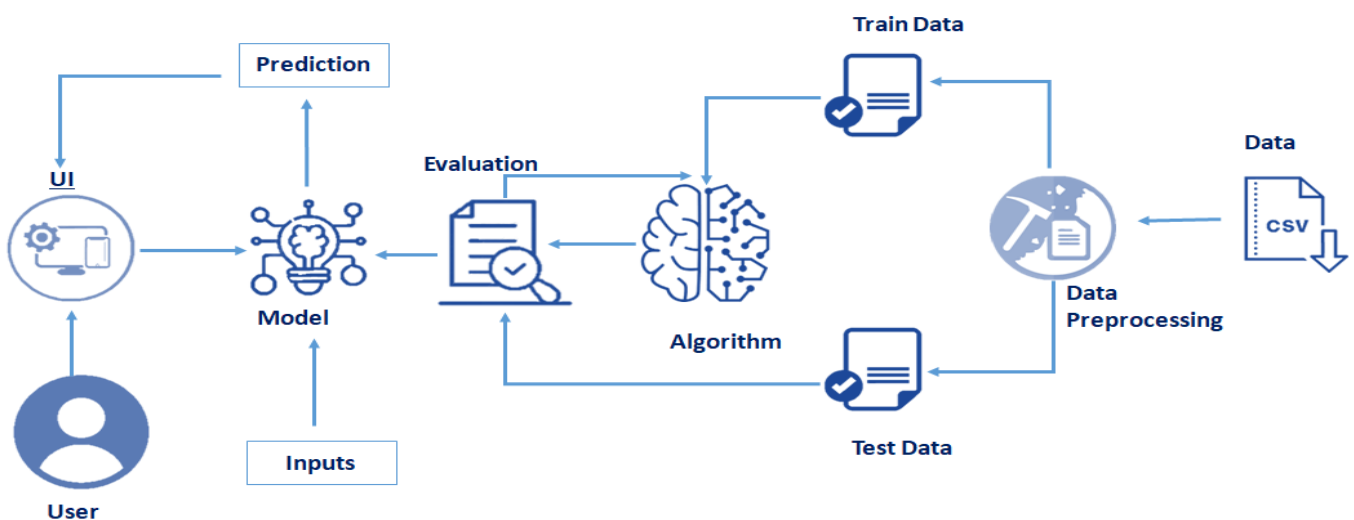
TrafficTelligence is an advanced system that uses machine learning algorithms to estimate and predict traffic volume with precision. By analysing historical traffic data, weather patterns, events, and other relevant factors, TrafficTelligence provides accurate forecasts and insights to enhance traffic management, urban planning, and commuter experiences.

Scenario 1: Dynamic Traffic Management TrafficTelligence enables dynamic traffic management by providing real-time traffic volume estimations. Transportation authorities can use this information to implement adaptive traffic control systems, adjust signal timings, and optimize lane configurations to reduce congestion and improve traffic flow.

Scenario 2: Urban Development Planning City planners and urban developers can leverage TrafficTelligence predictions to plan new infrastructure projects effectively. By understanding future traffic volumes, they can design road networks, public transit systems, and commercial zones that are optimized for traffic efficiency and accessibility.

Scenario 3: Commuter Guidance and Navigation Individual commuters and navigation apps can benefit from TrafficTelligence's accurate traffic volume estimations. Commuters can plan their routes intelligently, avoiding congested areas and selecting optimal travel times based on predicted traffic conditions. Navigation apps can provide real-time updates and alternative routes to improve overall travel experiences.

Technical Architecture



2. PRE REQUISITES

To complete this project, you must require the following software's, concepts, and packages.

Anaconda navigator:

Refer to the link below to download anaconda navigator.

https://youtu.be/5mDYijMfSzs?si=b_FTKtzvC-g790K

Python packages:

Open anaconda prompt as administrator.

- Type “pip install numpy” and click enter.
- Type “pip install pandas” and click enter.
- Type “pip install matplotlib” and click enter.
- Type “pip install scikit-learn” and click enter.
- Type “pip install Flask” and click enter.
- Type “pip install xgboost” and click enter.

3. PROJECT OBJECTIVES

By the end of this project:

- You'll be able to understand the problem to classify if it is a regression or a classification kind of problem.
- You will be able to know how to pre-process/clean the data using different data pre-processing techniques.
- You will be able to analyse or get insights into data through visualization.
- Applying different algorithms according to a dataset and based on visualization.
- You will be able to know how to find the accuracy of the model.
- You will be able to know how to build a web application using the Flask framework.

4. PROJECT FLOW

- User interacts with the UI (User Interface) to enter the input values.
- Entered input values are analysed by the model which is integrated.
- Once the model analyses the input the prediction is showcased on the UI.

To accomplish this, we have to complete all the activities and tasks listed below:

- Data Collection.
 - Collect the dataset or create the dataset
- Data Pre-processing.
 - Import the Libraries.
 - Importing the dataset.
 - Checking for Null Values.
 - Data Visualization.
 - Taking care of Missing Data.
 - Feature Scaling.
 - Splitting Data into Train and Test.
- Model Building
 - Import the model building Libraries
 - Initializing the model
 - Training and testing the model
 - Evaluation of Model
 - Save the Model
- Application Building
 - Create an HTML file
 - Build a Python Code
 - Run the App

5. PROJECT STRUCTURE

Create a Project folder that contains files as shown below:

Name	Type	Date Modified
> .ipynb_checkpoints	File Folder	11-12-2021 13:07
✓ Flask	File Folder	10-02-2022 11:35
> templates	File Folder	10-02-2022 11:35
app.py	py File	10-02-2022 11:35
01 encoder.pkl	pkl File	11-12-2021 13:01
01 model.pkl	pkl File	11-12-2021 13:01
✓ IBM	File Folder	15-02-2022 14:58
> Flask	File Folder	31-01-2022 10:30
traffic volume_ibm_scoring end point.ipynb	ipynb File	31-01-2022 10:27
Requirements.txt	txt File	15-12-2021 10:57
Traffic volume estimation.docx	docx File	15-02-2022 15:04
traffic volume.csv	csv File	10-12-2021 11:08
traffic volume.ipynb	ipynb File	11-12-2021 13:03

- Flask files consist of template folder which has HTML pages, app.py file and .pkl files which are used for application building.
- IBM folder has flask files and scoring endpoint.ipynb- model training code file.
- We need the model which is saved and the saved model in this content is Traffic volume. Pkl
- Templates folder which contains index.HTML file, chance.HTML file, noChance.HTML file.
- Scale.pkl for scaling, encoder.pkl file for encoding the categorical data, imputer.pkl file for filling out the missing values.

6. DATA COLLECTION

ML depends heavily on data, without data, it is impossible for an “AI” model to learn. It is the most crucial aspect that makes algorithm training possible. In Machine Learning projects, we need a training **data set**. It is the actual **data set** used to train the model for performing various actions.

Download The Dataset:

You can collect datasets from different open sources like kaggle.com, data.gov, UCI machine learning repository etc.

Please refer to the [link](#) given below to download the data set and to know about the dataset.

7. DATA PRE-PROCESSING

Data Pre-processing includes the following main tasks

- Import the Libraries.
- Importing the dataset.
- Checking for Null Values.
- Data Visualization.
- Feature Scaling.
- Splitting Data into Train and Test.

Import Necessary Libraries:

It is important to import all the necessary libraries such as pandas, NumPy, matplotlib.

- **Numpy**- It is an open-source numerical Python library. It contains a multi-dimensional array and matrix data structures. It can be used to perform mathematical operations on arrays such as trigonometric, statistical, and algebraic routines.
- **Pandas**- It is a fast, powerful, flexible and easy to use open-source data analysis and manipulation tool, built on top of the Python programming language.
- **Seaborn**- Seaborn is a Python data visualization library based on matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics.
- **Matplotlib**- Visualisation with python. It is a comprehensive library for creating static, animated, and interactive visualizations in Python
- **Sklearn** – which contains all the modules required for model building.

```
# importing the necessary libraries
import pandas as pd
import numpy as np
import seaborn as sns
import sklearn as sk
from sklearn import linear_model
from sklearn import tree
from sklearn import ensemble
from sklearn import svm
import xgboost
```

Importing The Dataset:

- You might have your data in .csv files, .excel files
- Let's load a .csv data file into pandas using read_csv() function. We will need to locate the directory of the CSV file at first (it's more efficient to keep the dataset in the same directory as your program).
- If your dataset is in some other location, Then
- **Data=pd.read_csv(r"File_location/datasetname.csv")**

```
# importing the data
data = pd.read_csv(r"G:\AI&ML\ML projects\Traffic_volume\traffic volume.csv")
```

Note: r stands for "raw" and will cause backslashes in the string to be interpreted as actual backslashes rather than special characters.

- If the dataset is in the same directory of your program, you can directly read it, without giving raw as r.
- Our Dataset traffic_data.csv contains the following Columns
- Holiday - working day or holiday
- Temp- temperature of the day
- Rain and snow – whether it is raining or snowing on that day or not
- Weather = describes the weather conditions of the day
- Date and time = represents the exact date and time of the day
- Traffic volume – output column

The output column to be predicted is Traffic volume. Based on the input variables we predict the volume of the traffic. The predicted output gives them a fair idea of the count of traffic.

Analyse The Data:

`head()` method is used to return top n (5 by default) rows of a DataFrame or series.

```
# displaying first 5 columns of the data  
data.head()
```

	holiday	temp	rain	snow	weather	date	Time	traffic_volume
0	None	288.28	0.0	0.0	Clouds	02-10-2012	09:00:00	5545
1	None	289.36	0.0	0.0	Clouds	02-10-2012	10:00:00	4516
2	None	289.58	0.0	0.0	Clouds	02-10-2012	11:00:00	4767
3	None	290.13	0.0	0.0	Clouds	02-10-2012	12:00:00	5026
4	None	291.14	0.0	0.0	Clouds	02-10-2012	13:00:00	4918

`describe()` method computes a summary of statistics like count, mean, standard deviation, min, max, and quartile values.

```
data.describe()
```

The output is as shown below

```
# used to understand the descriptive analysis of the data  
data.describe()
```

	temp	rain	snow	traffic_volume
count	48151.000000	48202.000000	48192.000000	48204.000000
mean	281.205351	0.334278	0.000222	3259.818355
std	13.343675	44.790062	0.008169	1986.860670
min	0.000000	0.000000	0.000000	0.000000
25%	272.160000	0.000000	0.000000	1193.000000
50%	282.460000	0.000000	0.000000	3380.000000
75%	291.810000	0.000000	0.000000	4933.000000
max	310.070000	9831.300000	0.510000	7280.000000

From the data, we infer that there are only decimal values and no categorical values.

`info()` gives information about the data - paste the image here.

```
# used to display the basic information of the data
```

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48204 entries, 0 to 48203
Data columns (total 8 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   holiday         48204 non-null  object
 1   temp            48151 non-null  float64
 2   rain            48202 non-null  float64
 3   snow            48192 non-null  float64
 4   weather         48155 non-null  object
 5   date            48204 non-null  object
 6   Time            48204 non-null  object
 7   traffic_volume  48204 non-null  int64
dtypes: float64(3), int64(1), object(4)
memory usage: 2.9+ MB
```

Handling Missing Values:

1. The Most important step in data pre-processing is dealing with missing data, the presence of missing data in the dataset can lead to low accuracy.
2. Check whether any null values are there or not. if it is present then the following can be done.

```
# used to display the null values of the data
```

```
data.isnull().sum()
```

```
holiday      0
temp         53
rain          2
snow         12
weather      49
date          0
Time          0
traffic_volume  0
dtype: int64
```

There are missing values in the dataset, we will fill the missing values in the columns.

3. We are using mean and mode methods for filling the missing values
 - Columns such as temp, rain, and snow are the numeric columns, when there is a numeric column you should fill the missing values with the mean/median method. so here we are using the mean method to fill the missing values.
 - Weather column has a categorical data type, in such case missing data needs to be filled with the most repeated/ frequent value. Clouds are the most repeated value in the column, so imputing with clouds value.

```
data['temp'].fillna(data['temp'].mean(),inplace=True)
data['rain'].fillna(data['rain'].mean(),inplace=True)
data['snow'].fillna(data['snow'].mean(),inplace=True)

print(Counter(data['weather']))

Counter({'Clouds': 15144, 'Clear': 13383, 'Mist': 5942, 'Rain': 5665, 'Snow': 2875, 'Drizzle': 1818, 'H
'Fog': 912, nan: 49, 'Smoke': 20, 'Squall': 4})

data['weather'].fillna('Clouds',inplace=True)
```

Data Visualization:

Data visualization is where a given data set is presented in a graphical format. It helps the detection of patterns, trends and correlations that might go undetected in text-based data.

Understanding your data and the relationship present within it is just as important as any algorithm used to train your machine learning model. In fact, even the most sophisticated machine learning models will perform poorly on data that wasn't visualized and understood properly.

- To visualize the dataset, we need libraries called Matplotlib and Seaborn.
- The Matplotlib library is a Python 2D plotting library that allows you to generate plots, scatter plots, histograms, bar charts etc.

Let's visualize our data using Matplotlib and seaborn library.

Before diving into the code, let's look at some of the basic properties we will be using when plotting.

xlabel: Set the label for the x-axis.

ylabel: Set the label for the y-axis.

title: Set a title for the axes.

Legend: Place a legend on the axes.

1. data.corr() gives the correlation between the columns

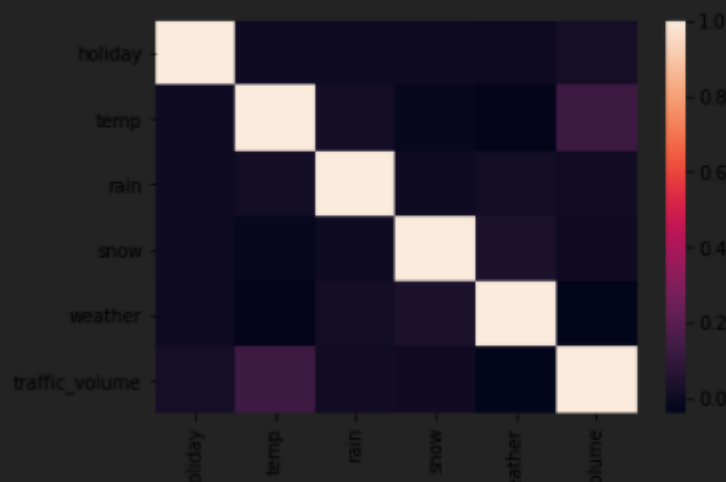
Correlation is a statistical term describing the degree to which two variables move in coordination with one another. If the two variables move in the same direction, then those variables are said to have a positive correlation. If they move in opposite directions, then they have a negative correlation.

```
cor
```

	holiday	temp	rain	snow	weather	traffic_volume
holiday	1.000000	-0.000472	0.000066	0.000432	-0.004328	0.018676
temp	-0.000472	1.000000	0.009070	-0.019758	-0.033559	0.130034
rain	0.000066	0.009070	1.000000	-0.000090	0.009542	0.004714
snow	0.000432	-0.019758	-0.000090	1.000000	0.036662	0.000735
weather	-0.004328	-0.033559	0.009542	0.036662	1.000000	-0.040035
traffic_volume	0.018676	0.130034	0.004714	0.000735	-0.040035	1.000000

```
sns.heatmap(cor)
```

```
<AxesSubplot:>
```



- Correlation strength varies based on colour, lighter the colour between two variables, more the strength between the variables, darker the colour displays the weaker correlation
- We can see the correlation scale values on the left side of the above image

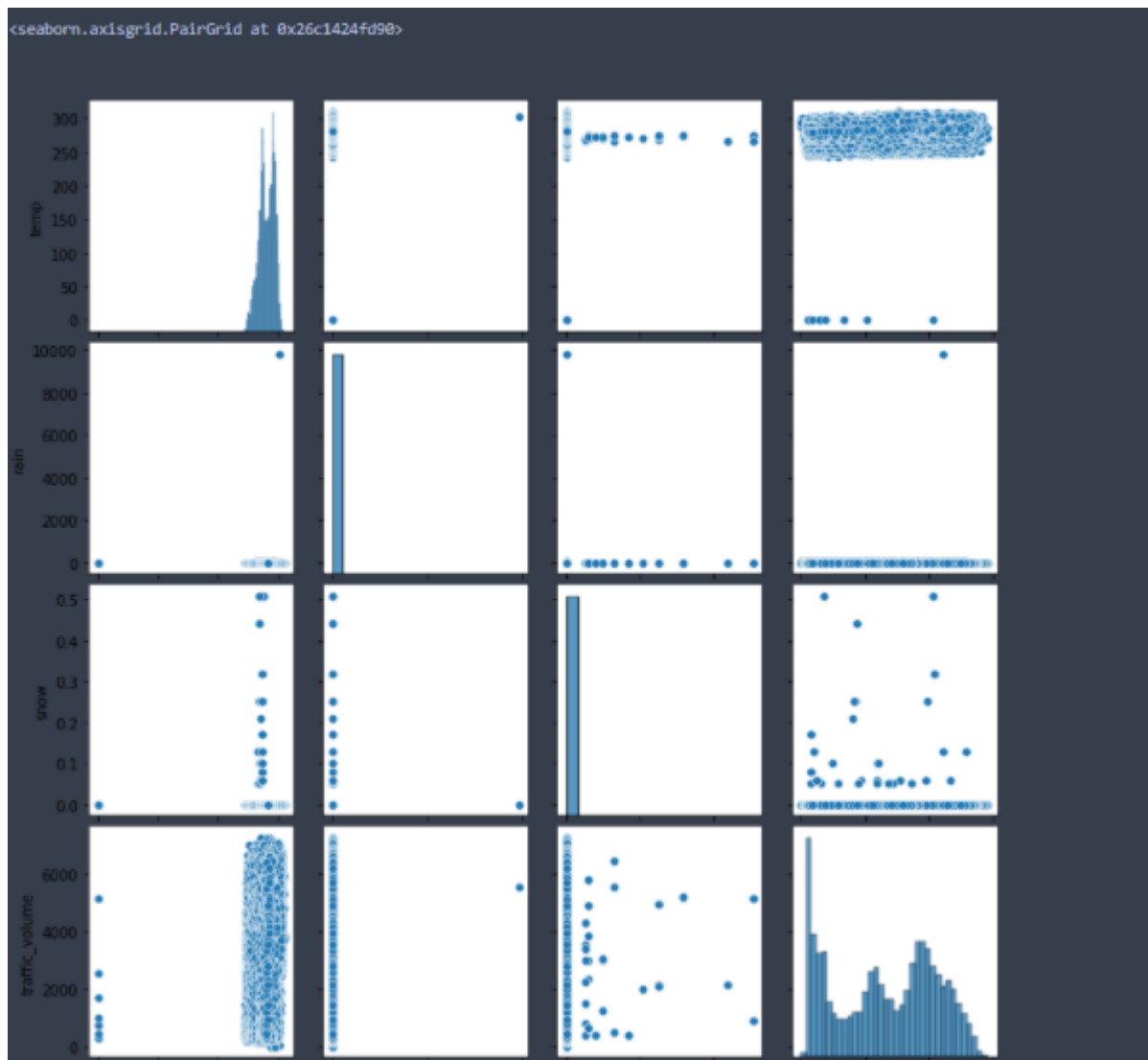
2. Pair Plot: Plot pairwise relationships in a dataset.

A pair plot is used to understand the best set of features to explain a relationship between two variables or to form the most separated clusters. It also helps to form some simple classification models by drawing some simple lines or making a linear separation in our data-set.

- By default, this function will create a grid of Axes such that each numeric variable in data will be shared across the y-axes across a single row and the x-axes across a single column. The diagonal plots are treated differently: a univariate distribution plot is drawn to show the marginal distribution of the data in each column.
- We implement this using the below code.

Code:- sns.pairplot(data)

The output is as shown below-



Pair plot usually gives pairwise relationships of the columns in the dataset

From the above pair plot, we infer that

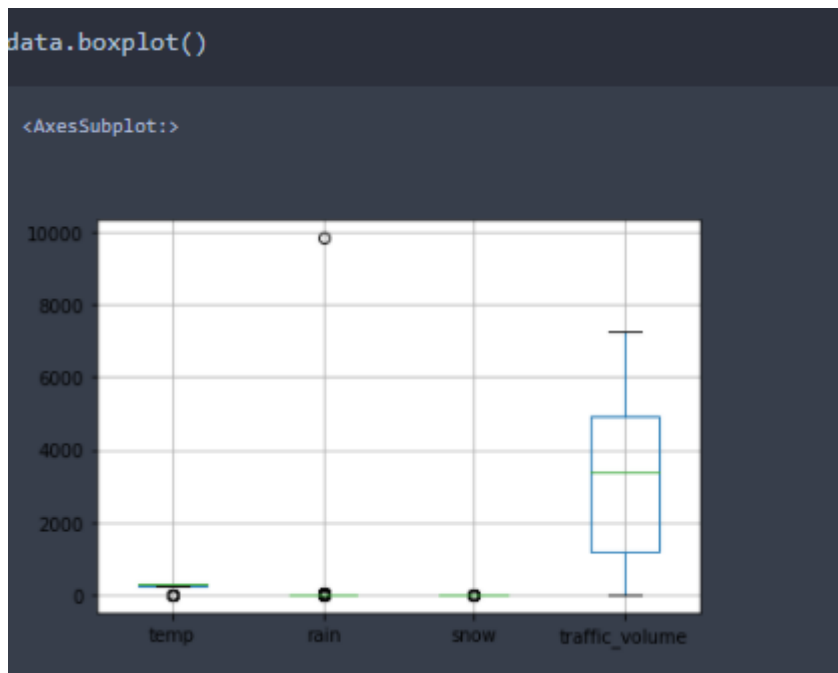
1. From the above plot we can draw inferences such as linearity and strength between the variables. how features are correlated (positive, neutral and negative)

3. Box Plot:

Box-plot is a type of chart often used in explanatory data analysis. Box plots visually show the distribution of numerical data and skewness through displaying the data quartiles (or percentiles) and averages.

Box plots are useful as they show the average score of a data set. The median is the average value from a set of data and is shown by the line that divides the box into two parts. Half the scores are greater than or equal to this value and half are less.

jupyter has a built-in function to create a boxplot called `boxplot()`. A boxplot plot is a type of plot that shows the spread of data in all the quartiles.



From the above box plot, we infer how the data points are spread and the existence of the outliers

4. Data and time columns need to be split into columns so that analysis and training of the model can be done in an easy way, so we use the split function to convert date into the year, month and day. time column into hours, minutes and seconds.

```
# splitting the date column into year,month,day
data[["day", "month", "year"]] = data["date"].str.split("-", expand = True)

# splitting the date column into year,month,day
data[["hours", "minutes", "seconds"]] = data["Time"].str.split(":", expand = True)

data.drop(columns=['date', 'Time'],axis=1,inplace=True)

data.head()
```

	holiday	temp	rain	snow	weather	traffic_volume	day	month	year	hours	minutes	seconds
0	7	288.28	0.0	0.0	1	5545	02	10	2012	09	00	00
1	7	289.36	0.0	0.0	1	4516	02	10	2012	10	00	00
2	7	289.58	0.0	0.0	1	4767	02	10	2012	11	00	00
3	7	290.13	0.0	0.0	1	5026	02	10	2012	12	00	00
4	7	291.14	0.0	0.0	1	4918	02	10	2012	13	00	00

Splitting The Dataset into Dependent and Independent Variable:

- In machine learning, the concept of the dependent variable (y) and independent variables(x) is important to understand. Here, the Dependent variable is nothing but output in dataset and the independent variable is all inputs in the dataset.
- With this in mind, we need to split our dataset into the matrix of independent variables and the vector or dependent variable. Mathematically, Vector is defined as a matrix that has just one column.

To read the columns, we will use iloc of pandas (used to fix the indexes for selection) which takes two parameters — [row selection, column selection].

Let's split our dataset into independent and dependent variables.

```
y = data[traffic_volume] - independent
```

```
x = data.drop(traffic_volume,axis=1)
```

```
y = data['traffic_volume']  
x = data.drop(columns=['traffic_volume'],axis=1)
```

Feature Scaling:

There is a huge disparity between the x values so let us use feature scaling.

Feature scaling is a method used to normalize the range of independent variables or features of data.

```

y = data['traffic_volume']
x = data.drop(columns=['traffic_volume'],axis=1)

names = x.columns

from sklearn.preprocessing import scale

x = scale(x)

x = pd.DataFrame(x,columns=names)

x.head()

```

	holiday	temp	rain	snow	weather	day	month	year	hours	minutes	seconds
0	0.015856	0.530485	-0.007463	-0.027235	-0.566452	-1.574903	1.02758	-1.855294	-0.345548	0.0	0.0
1	0.015856	0.611467	-0.007463	-0.027235	-0.566452	-1.574903	1.02758	-1.855294	-0.201459	0.0	0.0
2	0.015856	0.627964	-0.007463	-0.027235	-0.566452	-1.574903	1.02758	-1.855294	-0.057371	0.0	0.0
3	0.015856	0.669205	-0.007463	-0.027235	-0.566452	-1.574903	1.02758	-1.855294	0.086718	0.0	0.0
4	0.015856	0.744939	-0.007463	-0.027235	-0.566452	-1.574903	1.02758	-1.855294	0.230807	0.0	0.0

- After scaling the data will be converted into an array form
- Loading the feature names before scaling and converting them back to data frame after standard scaling is applied

Splitting The Data into Train and Test:

When you are working on a model and you want to train it, you obviously have a dataset. But after training, we have to test the model on some test datasets. For this, you will a dataset which is different from the training set you used earlier. But it might not always be possible to have so much data during the development phase. In such cases, the solution is to split the dataset into two sets, one for training and the other for testing.

- The train-test split is a technique for evaluating the performance of a machine learning algorithm.
- Train Dataset: Used to fit the machine learning model.
- Test Dataset: Used to evaluate the fit machine learning model.
- In general you can allocate 80% of the dataset to the training set and the remaining 20% to test.

- Now split our dataset into train set and test using train_test_split class from sci-kit learn library.

```
from sklearn import model_selection
```

```
x_train,x_test,y_train,y_test=model_selection.train_test_split(x,y,test_size=0.2,random_state=0)
```

```
from sklearn.model_selection import train_test_split
```

```
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2,random_state=0)
```

8. MODEL BUILDING

The model building includes the following main tasks

- Import the model building Libraries
- Initializing the model
- Training and testing the model
- Evaluation of Model
- Save the Model

Training and Testing the Model:

- Once after splitting the data into train and test, the data should be fed to an algorithm to build a model.
- There are several Machine learning algorithms to be used depending on the data you are going to process such as images, sound, text, and numerical values. The algorithms that you can choose according to the objective that you might have it may be Classification algorithms are Regression algorithms.
 1. Linear Regression
 2. Decision Tree Regressor
 3. Random Forest Regressor
 4. KNN
 5. svm
 5. xgboost

Steps in Building the model:

Initialize the model –

```
from sklearn import linear_model
from sklearn import tree
from sklearn import ensemble
from sklearn import svm
import xgboost

lin_reg = linear_model.LinearRegression()
Dtree = tree.DecisionTreeRegressor()
Rand = ensemble.RandomForestRegressor()
svr = svm.SVR()
XGB = xgboost.XGBRegressor()
```

Fit the models with x_train and y_train –

```
lin_reg.fit(x_train,y_train)
Dtree.fit(x_train,y_train)
Rand.fit(x_train,y_train)
svr.fit(x_train,y_train)
XGB.fit(x_train,y_train)
```

Predict the y_train values and calculate the accuracy –

```
p1 = lin_reg.predict(x_train)
p2 = Dtree.predict(x_train)
p3 = Rand.predict(x_train)
p4 = svr.predict(x_train)
p5 = XGB.predict(x_train)
```

We're going to use the x-train and y-train obtained above in the train_test_split section to train our Random forest regression model. We're using the fit method and passing the parameters as shown below.

We are using the algorithm from Scikit learn library to build the model as shown below, Once the model is trained, it's ready to make predictions. We can use the predict method on the model and pass x_test as a parameter to get the output as y_pred.

Notice that the prediction output is an array of real numbers corresponding to the input array.

Model Evaluation:

After training the model, the model should be tested by using the test data which is been separated while splitting the data for checking the functionality of the model.

Regression Evaluation Metrics:

These model evaluation techniques are used to find out the accuracy of models built in the Regression type of machine learning models. We have three types of evaluation methods.

- R-square_score
- RMSE – root mean squared error

1. R-squared _score -

Formula

$$R^2 = 1 - \frac{RSS}{TSS}$$

R^2 = coefficient of determination

RSS = sum of squares of residuals

TSS = total sum of squares

It is the ratio of the number of correct predictions to the total number of input samples.

Calculating the r2 score value using for all the models.

```
from sklearn import metrics
```

```
print(metrics.r2_score(p1,y_train))
print(metrics.r2_score(p2,y_train))
print(metrics.r2_score(p3,y_train))
print(metrics.r2_score(p4,y_train))
print(metrics.r2_score(p5,y_train))
```

```
-5.517285423636865
1.0
0.9747969952887571
-12.188104231382285
0.8349874938269883
```

```
p2 = Dtree.predict(x_test)
p3 = Rand.predict(x_test)
p4 = svr.predict(x_test)
p5 = XGB.predict(x_test)
```

```
print(metrics.r2_score(p1,y_test))
print(metrics.r2_score(p2,y_test))
print(metrics.r2_score(p3,y_test))
print(metrics.r2_score(p4,y_test))
print(metrics.r2_score(p5,y_test))
```

```
-5.399396398322181
0.6920677009517378
0.8031828166614183
-11.972215715232434
0.7922184852381723
```

- After considering both r squared values of test and train we concluded that random forest regressor is giving the better value, it is able to explain the 97% of the data in train values.
- Random forest gives the best r2-score, so we can select this model.

RMSE –Root Mean Square Error

Formula

$$\text{RMSE} = \sqrt{\frac{\sum_{i=1}^N (x_i - \hat{x}_i)^2}{N}}$$

RMSE = root-mean-square deviation

i = variable i

N = number of non-missing data points

x_i = actual observations time series

$\hat{x}_i$ = estimated time series

Randomforest gives the best r-score value

```
#RMSE values
MSE = metrics.mean_squared_error(p3,y_test)

np.sqrt(MSE)

798.4970439382182
```

RMSE value for Random forest is very less when compared with other models, so saving the Random forest model and deploying using the following process

Save The Model:

After building the model we have to save the model.

Pickle in Python is primarily used in serializing and deserializing a Python object structure. In other words, it's the process of converting a Python object into a byte stream to store it in

a file/database, maintain program state across sessions or transport data over the network. wb indicates write method and rd indicates read method.

This is done by the below code

```
import pickle

pickle.dump(Rand,open("model.pkl",'wb'))
pickle.dump(le,open("encoder.pkl",'wb'))
```

9. APPLICATION BUILDING

In this section, we will be building a web application that is integrated into the model we built. A UI is provided for the uses where he has to enter the values for predictions. The enter values are given to the saved model and prediction is showcased on the UI.

This section has the following tasks

- Building HTML Pages
- Building server-side script

Build HTML Code:

In this HTML page, we will create the front-end part of the web page. On this page, we will accept input from the user and Predict the values.

For more information regarding HTML, click on the link.

In our project we have HTML files, they are

1.index.html - paste the image

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="UTF-8">
5 <title>Traffic Volume Estimation</title>
6 </head>
7
8
9 <body background="https://cdn.vox-cdn.com/thumbor/VOAR3fE0V7p5lUSzW0EXpN6QTDW0/0/781:3000x1756/1600x900/cdn.vox-cdn.com/uploads/chorus_image/image/44219365/72499026_0_0.jpg" text="black">
10
11 <div class="Login">
12 <center><h1>Traffic Volume Estimation</h1></center>
13
14 <!-- Main Input For Receiving Query to our ML -->
15 <form action="{ url_for('predict') }}" method="post">
16 <h1>Please enter the following details</h1>
17
18
19 </style></head>
20
21
22 <label for="holiday">holiday:</label>
23 <select id="holiday" name="holiday">
24 <option value=7>None</option>
25 <option value=1>Columbus Day</option>
26 <option value=10>Veterans Day</option>
27 <option value=9>Thanksgiving Day</option>
28 <option value=0>Christmas Day</option>
29 <option value=6>New Years Day</option>
30 <option value=11>Washingtons Birthday</option>
31 <option value=5>Memorial Day</option>
32 <option value=2>Independence Day</option>
33 <option value=8>State Fair</option>
34 <option value=3>Labor Day</option>
35 <option value=4>Martin Luther King Jr Day</option>
36
37 </select> &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;<br>
38
39
40 <br> <label>temp:</label>
41 <input type="number" name="temp" placeholder="temp " required="required" /><br>
42
43 <br>
44 <label>rain:</label>
45 <input type="number" min="0" max="1" name="rain " placeholder="rain " required="required" /><br>
46
47 <br>
48 <label>snow:</label>
49

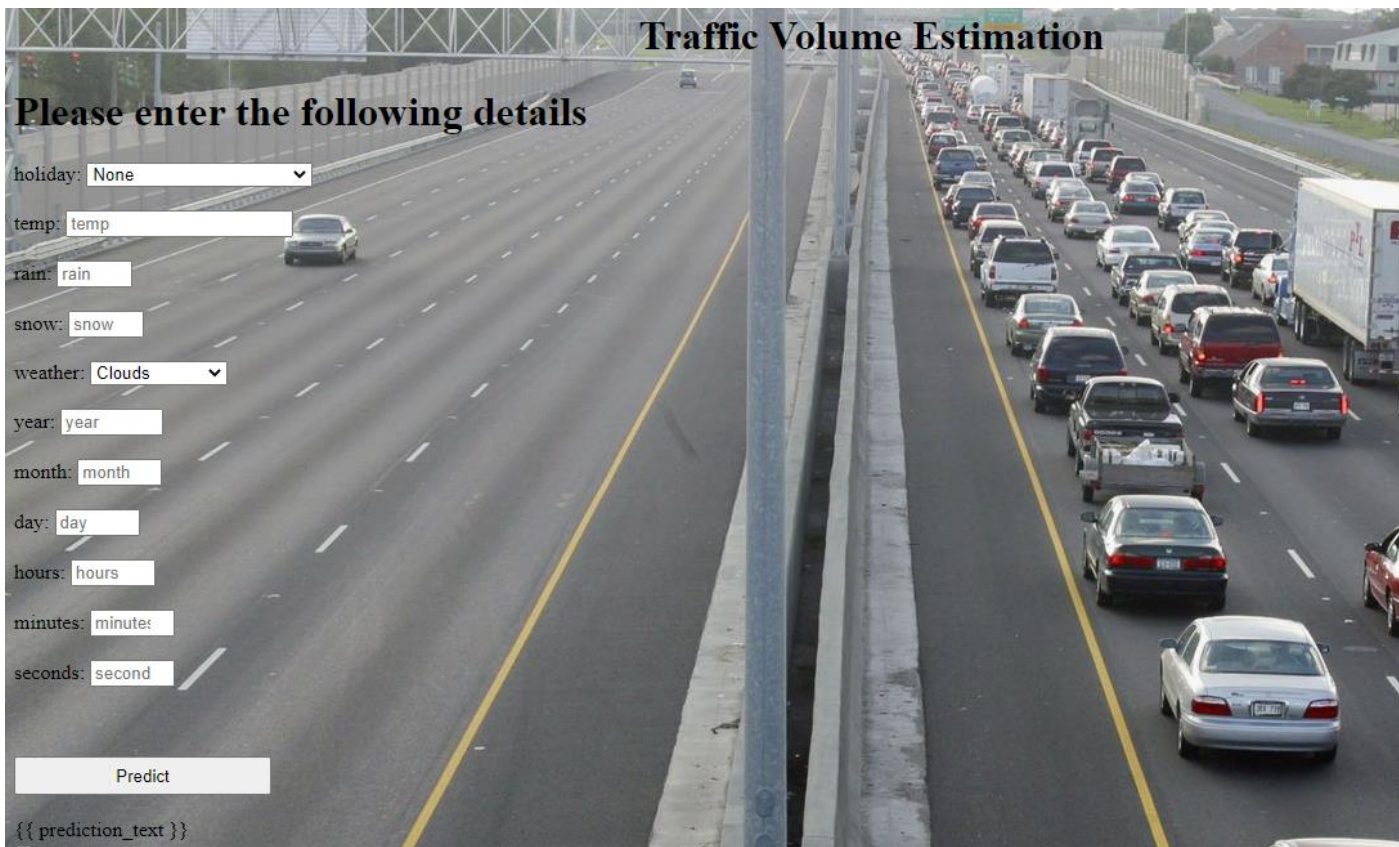
```

```

8 <br>
9 <label>snow:</label>
10 <input type="number" min="0" max="1" name="snow" placeholder="snow" required="required" /><br>
11
12 <br>
13
14 <label for="weather">weather:</label>
15 <select id="weather" name="weather">
16 <option value=1>Clouds</option>
17 <option value=0>Clear</option>
18 <option value=6>Rain</option>
19 <option value=2>Drizzle</option>
20 <option value=5>Mist</option>
21 <option value=4>Haze</option>
22 <option value=3>Fog</option>
23 <option value=10>Thunderstorm</option>
24 <option value=8>Snow</option>
25 <option value=9>Squall</option>
26 <option value=7>Smoke</option><
27
28 </select> &nbsp;&nbsp;&nbsp;<br>
29
30 <br>
31 <label>year:</label>
32 <input type="number" min="2012" max="2022" name="year" placeholder="year" required="required" /><br>
33
34 <br>
35 <label>month:</label>
36 <input type="number" min="1" max="12" name="month" placeholder="month" required="required" /><br>
37
38 <br>
39 <label>day:</label>
40 <input type="number" min="1" max="31" name="day" placeholder="day" required="required" /><br>
41
42 <br>
43 <label>hours:</label>
44 <input type="number" min="0" max="24" name="hours" placeholder="hours" required="required" /><br>
45
46 <br>
47 <label>minutes:</label>
48 <input type="number" min="0" max="60" name="minutes" placeholder="minutes" required="required" /><br>
49
50 <br>
51 <label>seconds:</label>
52 <input type="number" min="0" max="60" name="seconds" placeholder="seconds" required="required" /><br>
53
54 <br>
55 <br><br>
56 <button type="submit" class="btn btn-primary btn-block btn-large" style="height:30px;width:200px">Predict</button>
57
58 </form>
59 <br>
60
61 {{ prediction_text }}
62
63 <br>
64 <br>
65 

```

2. The HTML page looks like this-



Traffic Volume Estimation

Please enter the following details

holiday: None

temp: temp

rain: rain

snow: snow

weather: Clouds

year: year

month: month

day: day

hours: hours

minutes: minute

seconds: second

Predict

{{ prediction_text }}

3. It will display all the input parameters and the prediction text will display the output value of the data given by the user.



Main Python Script:

Let us build an app.py flask file which is a web framework written in python for server-side scripting. Let's see step by step procedure for building the backend application.

In order to develop web API with respect to our model, we basically use the Flask framework which is written in python.

Line 1-9 We are importing necessary libraries like Flask to host our model request

Line 12 Initialise the Flask application

Line 13 Loading the model using pickle

Line 16 Routes the API URL

Line 18 Rendering the template. This helps to redirect to the home page. In this home page, we give our input and ask the model to predict

In line 23 we are taking the inputs from the form

Line 28 Feature Scaling the inputs

Line 31 Predicting the values given by the user

Line 32-35 if the output is false render no chance template If the output is True render chance template

Line 36 The value of `__name__` is set to `__main__` when the module run as the main program otherwise it is set to the name of the module .

```
1  import numpy as np
2  import pickle
3  import joblib
4  import matplotlib
5  import matplotlib.pyplot as plt
6  import time
7  import pandas
8  import os
9  from flask import Flask, request, jsonify, render_template
10
11
12  app = Flask(__name__)
13  model = pickle.load(open('G:/AI&ML/ML projects/Traffic_volume/model.pkl', 'rb'))
14  scale = pickle.load(open('C:/Users/SmartbridgePC/Desktop/AI/ML/Guided projects/scale.pkl', 'rb'))
15
16  @app.route('/')# route to display the home page
17  def home():
18      return render_template('index.html') #rendering the home page
19
20  @app.route('/predict',methods=["POST","GET"])# route to show the predictions in a web UI
21  def predict():
22      # reading the inputs given by the user
23      input_feature=[float(x) for x in request.form.values() ]
24      features_values=[np.array(input_feature)]
25      names = [['holiday', 'temp', 'rain', 'snow', 'weather', 'year', 'month', 'day',|
26              'hours', 'minutes', 'seconds']]
27      data = pandas.DataFrame(features_values,columns=names)
28      data = scale.fit_transform(data)
29      data = pandas.DataFrame(data,columns = names)
30      # predictions using the loaded model file
31      prediction=model.predict(data)
32      print(prediction)
33      text = "Estimated Traffic Volume is :"
34      return render_template("index.html",prediction_text = text + str(prediction))
35      # showing the prediction results in a UI
36  if __name__=="__main__":
37
38      # app.run(host='0.0.0.0', port=8000,debug=True)      # running the app
39      port=int(os.environ.get('PORT',5000))
40      app.run(port=port,debug=True,use_reloader=False)
```

Run The App:

Open anaconda prompt from the start menu

- Navigate to the folder where your python script is.
- Now type the “python app.py” command

Navigate to the localhost where you can view your web page, Then it will run on **local host:5000**

```
In [1]: runfile('G:/AI&ML/ML projects/Traffic_volume/app.py', wdir='G:/AI&ML/ML
projects/Traffic_volume')
* Serving Flask app "app" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: on
/Users/SmartbridgePC/anaconda3/lib/site-packages/sklearn/base.py:324:
UserWarning: Trying to unpickle estimator StandardScaler from version 0.23.2 when
loading version 1.0.1. This might lead to breaking code or invalid results. Use at
your own risk. For more info please refer to:
https://scikit-learn.org/stable/modules/model_persistence.html#security-
sustainability-limitations
  warnings.warn(
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

Output:

- Copy the HTTP link and paste it in google link tab, it will display the form page
- Enter the values as per the form and click on predict button
- It will redirect to the page based on prediction output
- The output will be displayed in the prediction text as Estimated Traffic volume is in units.



10.CONCLUSION

The project “**TrafficTelligence: Advanced Traffic Volume Estimation Using Machine Learning**” has provided a practical and insightful approach to addressing the real-world challenge of traffic management. By leveraging supervised machine learning algorithms such as **Random Forest Regressor** and **XGBoost Regressor**, the system effectively analyzes historical traffic data combined with environmental features like temperature, rainfall, snowfall, date, and time to estimate traffic volume with considerable accuracy.

Throughout the development process, we focused on data preprocessing, feature engineering, model training, evaluation, and deployment using a Flask web application. The final model demonstrated strong performance with high **R² scores and low RMSE**, indicating its effectiveness in predicting traffic volume trends. This information can be a valuable asset for **urban planners, commuters, traffic authorities, and navigation systems**, helping them make smarter, real-time decisions.

This project also helped us gain hands-on experience in **data science, machine learning workflows, and web application deployment**, enhancing both our technical and problem-solving abilities. The seamless integration of data science with web technologies has opened up a pathway for developing intelligent solutions that contribute to **smart city development and sustainable urban mobility**.

In conclusion, TrafficTelligence represents a small but meaningful step toward building **data-driven traffic ecosystems** that are proactive, efficient, and scalable.